The Data Diode Handbook

Software and hardware unidirectional links,
and the proven OPC UA relay that runs across them

Alin-Adrian Anton
alin.anton@upt.ro
Computer and Information Technology Department
Politehnica University of Timişoara

For the opc-diode project
Diode build material shared across the sibling projects; canonical copy in log-diode

Abstract

A data diode enforces one-way information flow. This handbook covers how to build one, how to prove you have built one, and how to run useful traffic across it. It treats three families: **firewall-based diodes** (iptables, nftables, pf); a **unidirectional layer-2 tunnel** built by bridging a TAP interface to each of two real Ethernet segments and forwarding frames with a program that has no reverse code path; and **physical diodes** — commodity media converters between networks, or a three-wire SPI link between two microcontrollers. The first two are excellent bench instruments and are *not* security boundaries; only the third survives a compromise of the hosts it separates. For the optical case it works through both published topologies — two converters with a fiber splitter on the transmitter's return path, and three converters in a triangle — with the power budgets, the attenuation they imply, and the procurement reality that actually decides between them. It closes with the transport problem a diode creates, and with od_sender/od_receiver: Ada/SPARK daemons whose parsing and reconstruction path is machine-checked free of run-time errors, because on a link with no feedback there is no second chance.

Contents

1 Scope

1.1 What this document is

The *Sensors* paper [1] gives the essential recipe for the diode itself in its §2.2: static ARP, addressing, and two media-converter topologies. This handbook expands that recipe into something you can hand to an engineer, and adds three things the paper does not cover:

1. **Software diodes.** The paper does not discuss them. Both the firewall rulesets of §4 and the bridged layer-2 tunnel of §4.7 are genuinely useful — for development, CI and teaching — and genuinely dangerous if mistaken for the real thing, so they get an honest chapter with an honest warning.
2. **The optical practicalities.** §5.3 gives the power budgets, says when to reach for an attenuator, and is honest about the part that actually gates a splitter build — an 850 nm multimode splitter is a special order with a long lead time, while 1310 nm ones are catalogue stock.
3. **Embedded scale.** Between two microcontrollers a media converter is absurd. §5.8 builds the same guarantee from three wires of SPI, and explains why the sender — not the receiver — has to be the bus master.
4. **Commissioning.** How to *prove* a link is one-way, rather than believe it.

1.2 How to read it

The *Quick Start* companion is the build sheet; this is the reference. If you want a working diode today, read that first and come back here for the reasoning. Sections §2 and §6 are the ones to read before anything goes into production.

1.3 Conventions

Throughout, SOURCE is the network data leaves and DESTINATION is the network it arrives in. Information flows SOURCE → DESTINATION only.

These are *roles, not security levels*, and the distinction matters. A diode is deployed in one of two orientations, and which one you are building decides what the device is protecting:

- **Ingest** — the source is the lower-security network and the destination the higher-security one. The diode protects the destination's *confidentiality*: the untrusted side can send in but can never read back. This is the Bell–LaPadula arrangement [2], and it is how log collection works — a SOC ingests logs from a network where an attacker may already be.
- **Export** — the source is the higher-security network and the destination the lower-security one. The diode protects the source's *integrity and reachability*: the protected side publishes outward, and nothing on the receiving network can reach in, write to it, or probe it. This is how a plant exports process data or a database ships its write-ahead log to a mirror.

Everything mechanical in this handbook — the firewall rules, the media converters, the SPI link, the commissioning tests — is identical in both orientations, which is why it is written in terms of source and destination. §2.2 states which orientation this project uses.

Addresses are `192.168.10.1` (SOURCE) and `192.168.10.2` (DESTINATION); the diode-facing interface is `enp1s0` on Linux and `em0` on BSD.

2 Threat model and assurance tiers

2.1 The property being claimed

A data diode makes one claim: *no information can travel from* DESTINATION *back to* SOURCE. Everything else — confidentiality, integrity, reliability — is built on top of that claim by the software running across the link, not by the link itself.

That single claim is what both orientations rest on. In an ingest deployment it means the untrusted sender can never read the protected receiver, which is Bell–LaPadula confidentiality [2]. In an export deployment it means the receiving network can never reach back into the protected sender, which is an integrity and isolation property rather than a confidentiality one. The hardware is the same; only what you are protecting changes.

2.2 Orientation for this project

opc-diode is an export deployment. The source is the *higher*-security network: a plant publishes OPC UA PubSub (UADP over UDP) natively, and the diode carries those NetworkMessages out to subscribers on a lower-security monitoring network. So SOURCE = high security and DESTINATION = low security here — the reverse of an ingest deployment.

The property being bought is therefore not confidentiality of the receiver but **integrity and reachability of the sender**: the monitoring network can read process data, and can never reach back into the plant to write a setpoint, probe a PLC, or pivot. PubSub helps here, being connectionless and one-way by construction, so there is no session for the diode to break.

What a diode does not do

A diode says nothing about the *truthfulness* of what crosses it. A compromised device on SOURCE can send whatever it likes, including forged or replayed log lines. The diode guarantees only that the attacker cannot *see* or *reach* DESTINATION. Authenticity of content is the cryptography's job (§7), and the authenticity of the *sender* depends on the security of the SOURCE devices themselves.

2.3 Three tiers, and where each fails

Tier	Enforced by	How it fails
Application	A daemon that chooses not to reply	A bug, a configuration change, or code execution in the daemon. Offers essentially no assurance; not treated further here.
Firewall	A ruleset in the kernel	Root compromise, `nft flush ruleset`, a configuration-management run, a kernel or conntrack bug, an interface rename. Recovers a full return path instantly and silently.
Layer-2 tunnel (§4.7)	A forwarding program with no reverse code path	Replacing the binary, or bridging both NICs together. Stronger than a ruleset — the reverse path is absent rather than forbidden — but still one command away for root on the forwarding host.
Physical (optics, or SPI with MISO omitted)	There is no transmitter, and no conductor, pointed the wrong way	Someone rewires it, or fits the missing wire. Nothing software can do restores the path.

Table 1: Assurance tiers. Only the optical tier survives a compromise of the hosts it separates.

The distinction matters because of *who* you are defending against. If the adversary is a misconfigured application, a firewall diode helps. If the adversary has code execution on the SOURCE host — which is the entire premise of putting a diode between the log producers and the SOC — then the firewall is inside the attacker's blast radius and the guarantee evaporates. As

the paper puts it, the highest security diode enforces unidirectionality from physical properties and is immune from software exploits [1].

2.4 Residual risks even with optics

- **Rewiring.** A person with physical access can restore a return path in seconds. This is the paper's "unintentional insider": an administrator who tidies the cables. Mitigation is procedural — labelling, tamper-evident seals, locked enclosures, and the audit in §6.
- **Covert channels.** A colluding insider can exfiltrate through channels the diode does not model: NIC LEDs [6], Ethernet cable emissions [7], power lines [8], and memory-bus emissions picked up by a nearby phone [9]. A diode is not a TEMPEST control.
- **Loss.** A one-way link cannot retransmit. Loss beyond the error code's capacity is unrecoverable. This is the honest failure mode of a diode and is a design constraint, not a defect.
- **No diagnostics.** Nothing can report back. Failure is silent by construction; see §5.6.
- **Availability.** Every converter is a point of failure, and a three-converter diode has three of them.

3 Why the transport looks the way it does

Remove the return path and a surprising amount of ordinary networking stops working. It is worth being explicit, because every one of these becomes a configuration task.

What breaks	Consequence
TCP	No handshake, so no TCP at all. Everything must be UDP or a raw protocol.
ARP	The sender never learns the receiver's MAC. Must be configured statically — this is why `/etc/ethers` appears in every recipe here.
ICMP	No `ping`, no path MTU discovery. Set the MTU by hand and keep datagrams comfortably under it.
DNS, NTP, DHCP	All require a reply. Use static addressing, static `/etc/hosts`, and an DESTINATION-local time source.
Retransmission	Nothing can be requested again. Redundancy has to be sent *in advance*.
Key exchange	No Diffie–Hellman, no TLS. Keys are pre-shared out of band.

The last two shape the software. Since a lost datagram is lost forever, the sender must decide up front how much redundancy to spend. The paper's answer was a repeat factor: send each message R times. The tools described in §7 improve on it in two ways — a Reed–Solomon erasure code, so that any K of $K + M$ fragments rebuild the message rather than needing one intact copy; and interleaving, so that a burst of loss cannot consume every copy of the same message.

4 Software diodes

This entire section is for the bench

A firewall diode enforces direction with a mutable ruleset inside a kernel that the attacker may be able to reach. `nft flush ruleset` is one command. A configuration-management run that re-applies a site-wide policy is one mistake. Treat everything below as a **laboratory and CI instrument**: it lets you develop against od_sender/od_receiver, write integration tests, and teach the topology without buying hardware. It is not a boundary between two security domains. For that, build §5.

4.1 The one principle

Every ruleset in this section obeys a single negative rule:

Nothing on the diode interface may create or consult connection state.

A conntrack entry, an `ESTABLISHED,RELATED` accept, or a pf `keep state` exists precisely to let the reply through. So does a TCP RST and an ICMP port-unreachable — both are packets travelling the wrong way, and both leak information (they tell the DESTINATION side that something is listening). Stateless, silent dropping is the only correct behaviour.

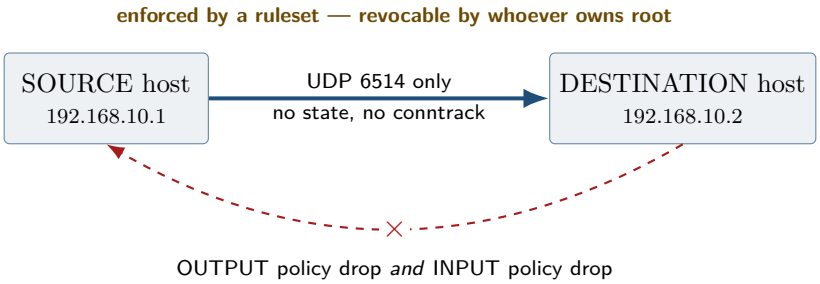

Figure 1: The software diode. The asymmetry is real but it lives in mutable kernel state.

4.2 Link-layer preparation (all platforms)

This layer is needed for hardware diodes too — with no return path, address resolution has to be done by hand.

4.2.1 Linux

Record the peer's MAC in `/etc/ethers` on each host (SOURCE records DESTINATION's, and vice versa):

```
# /etc/ethers on the SOURCE host
bb:bb:bb:bb:bb:bb 192.168.10.2
```

```
ip addr add 192.168.10.1/24 dev enp1s0     # .2/24 on the DESTINATION host
ip link set enp1s0 up
arp -f /etc/ethers                          # net-tools package
ip neigh show dev enp1s0                    # must report PERMANENT
```

The paper loads this from `/etc/rc.local` via a `rc-local.service` shim [1]; a dedicated unit is cleaner on a modern system:

```
[Unit]
Description=Static ARP and addressing for the diode link
After=network-pre.target
Wants=network-pre.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/sbin/arp -f /etc/ethers

[Install]
WantedBy=multi-user.target
```

Listing 1: `/etc/systemd/system/diode-link.service`

Then quieten the interface:

```
# Never answer an ARP request on the diode NIC; never volunteer our address.
net.ipv4.conf.enp1s0.arp_ignore   = 8
net.ipv4.conf.enp1s0.arp_announce = 2

# IPv6 is chatty and inherently bidirectional (NDP, RA). Turn it off here.
net.ipv6.conf.enp1s0.disable_ipv6 = 1

net.ipv4.conf.enp1s0.accept_redirects    = 0
net.ipv4.conf.enp1s0.send_redirects      = 0
net.ipv4.conf.enp1s0.accept_source_route = 0
```

Listing 2: `/etc/sysctl.d/90-diode.conf`

Reverse-path filtering

Leave `rp_filter` alone unless you know your topology needs it. In the symmetric-subnet layout used here, strict reverse-path filtering (`rp_filter=1`) passes, because the route to the peer's source address does point out of `enp1s0`. If you place the two sides in *different* subnets — a common choice, since routing back is impossible anyway — strict RPF will silently discard every arriving datagram on the DESTINATION host. That failure looks exactly like a broken diode. Set `net.ipv4.conf.enp1s0.rp_filter = 0` in that case and note it in the commissioning record.

If your distribution runs NetworkManager, exclude the diode interface so it does not re-apply DHCP or IPv6 autoconfiguration behind your back:

```
[keyfile]
unmanaged-devices=interface-name:enp1s0
```

Listing 3: `/etc/NetworkManager/conf.d/90-diode.conf`

4.2.2 BSD

```
ifconfig em0 inet 192.168.10.1/24 up
arp -s 192.168.10.2 bb:bb:bb:bb:bb:bb permanent
# persist: /etc/hostname.em0 for the address, /etc/rc.local for the arp entry
```

Listing 4: OpenBSD

```
ifconfig_em0="inet 192.168.10.1 netmask 255.255.255.0"
static_arp_pairs="diodepeer"
static_arp_diodepeer="192.168.10.2 bb:bb:bb:bb:bb:bb"
pf_enable="YES"
pf_rules="/etc/pf.conf"
```

Listing 5: FreeBSD `/etc/rc.conf`

4.3 nftables

nftables is the recommended Linux option, for one reason that iptables cannot match: the `netdev` family's `ingress` hook sees *every* frame, including ARP, LLDP and IPv6, before the IP stack does. A single `policy drop` there makes the interface deaf at the link layer.

```
#!/usr/sbin/nft -f
flush ruleset
```

```
define PEER = 192.168.10.2
define PORT = 6514

# Link-layer deafness. Note: the device name cannot be a variable in
# older nft releases, so it is spelled out here.
table netdev diodeguard {
    chain ingress {
        type filter hook ingress device enp1s0 priority -500; policy drop;
    }
}

table inet diode {
    chain input {
        type filter hook input priority filter; policy drop;
        iif lo accept
        iifname "enp1s0" counter drop
    }
    chain output {
        type filter hook output priority filter; policy drop;
        oif lo accept
        oifname "enp1s0" ip daddr $PEER udp dport $PORT counter accept
        oifname "enp1s0" counter drop
    }
    chain forward {
        type filter hook forward priority filter; policy drop;
    }
}
```

Listing 6: `/etc/nftables.conf` — SOURCE host (transmit only)

```
#!/usr/sbin/nft -f
flush ruleset

define PEER = 192.168.10.1
define PORT = 6514

table inet diode {
    chain input {
        type filter hook input priority filter; policy drop;
        iif lo accept
        iifname "enp1s0" ip saddr $PEER udp dport $PORT counter accept
        iifname "enp1s0" counter drop
    }
    chain output {
        type filter hook output priority filter; policy drop;
        oif lo accept
        # Absolutely nothing leaves by the diode NIC.
        oifname "enp1s0" counter drop
    }
    chain forward {
        type filter hook forward priority filter; policy drop;
    }
}
```

Listing 7: `/etc/nftables.conf` — DESTINATION host (receive only)

The `counter` keywords are not decoration. After a test run, `nft list ruleset` shows how many packets each rule matched; a non-zero counter on the DESTINATION host's output-drop rule is direct evidence that something tried to talk backwards.

```
systemctl enable --now nftables
nft list ruleset            # always read back what actually loaded
```

```
nft list ruleset | grep -c "ct state"   # must print 0
```

4.4 iptables

Still widespread, and named explicitly in many site standards. The rules are a direct transliteration, with one important gap.

```
iptables -F && iptables -X
iptables -P INPUT   DROP
iptables -P OUTPUT  DROP
iptables -P FORWARD DROP

iptables -A INPUT  -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

# Nothing arriving on the diode NIC is ever accepted.
iptables -A INPUT  -i enp1s0 -j DROP

# Exactly one thing may leave by it.
iptables -A OUTPUT -o enp1s0 -p udp -d 192.168.10.2 --dport 6514 -j ACCEPT
iptables -A OUTPUT -o enp1s0 -j DROP

# IPv6 off entirely.
ip6tables -P INPUT DROP; ip6tables -P OUTPUT DROP; ip6tables -P FORWARD DROP
```

Listing 8: SOURCE host

```
iptables -F && iptables -X
iptables -P INPUT   DROP
iptables -P OUTPUT  DROP
iptables -P FORWARD DROP

iptables -A INPUT  -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

iptables -A INPUT  -i enp1s0 -p udp -s 192.168.10.1 --dport 6514 -j ACCEPT
iptables -A INPUT  -i enp1s0 -j DROP
iptables -A OUTPUT -o enp1s0 -j DROP

ip6tables -P INPUT DROP; ip6tables -P OUTPUT DROP; ip6tables -P FORWARD DROP
```

Listing 9: DESTINATION host

iptables cannot filter ARP

iptables operates on IP. ARP traffic passes it untouched, so on iptables alone the DESTINATION host will still answer ARP requests across the link — a working, if narrow, return channel. Close it with arptables (or ebtables), or use nftables, whose `netdev` ingress hook covers all of it:

```
arptables -P INPUT DROP
arptables -P OUTPUT DROP
```

The `arp_ignore=8` sysctl in §4.2 is a second layer for the same problem, and should be applied regardless.

Note also that `-P OUTPUT DROP` is host-wide. If the machine has a management interface, add explicit accepts for it before you apply this over SSH.

4.5 pf (OpenBSD and FreeBSD)

pf makes the stateless requirement explicit and readable, and it has a setting that matters more here than anywhere else: `set block-policy drop`. The alternative, `return`, answers blocked traffic with TCP RSTs and ICMP unreachables — packets flowing the wrong way.

```
diode_if = "em0"
peer     = "192.168.10.2"
port     = 6514

# Silence. Never answer with an RST or an ICMP unreachable: that is a
# packet travelling from HIGH to LOW.
set block-policy drop
set skip on lo

block all

# The single permitted flow. "no state" is the whole point -- a state
# table entry exists to let the reply back in.
pass out quick on $diode_if inet proto udp \
     from ($diode_if) to $peer port $port no state

block in  quick on $diode_if all
block out quick on $diode_if all
```

Listing 10: `/etc/pf.conf` — SOURCE host

```
diode_if = "em0"
peer     = "192.168.10.1"
port     = 6514

set block-policy drop
set skip on lo

block all

pass in quick on $diode_if inet proto udp \
     from $peer to ($diode_if) port $port no state

block out quick on $diode_if all
```

Listing 11: `/etc/pf.conf` — DESTINATION host

```
pfctl -nf /etc/pf.conf      # syntax check first
pfctl -f  /etc/pf.conf
pfctl -sr                   # read back the loaded rules
pfctl -si | grep -i state   # state table should stay empty
```

OpenBSD versus FreeBSD. The syntax above works on both. Differences worth knowing: OpenBSD enables pf by default and loads `/etc/pf.conf` at boot, while FreeBSD needs `pf_enable="YES"` in `/etc/rc.conf`. FreeBSD's pf is a fork of an older OpenBSD release, so newer OpenBSD-only syntax will not parse there — everything used here predates the split. Interface names differ (`em0`, `igb0`, `re0`, `vtnet0`). FreeBSD users who prefer ipfw can express the same policy with `ipfw add deny ip from any to any in via em0` on the SOURCE host; the stateless requirement is identical — do not use `keep-state` or `check-state`.

4.6 A diode in network namespaces, for CI

The cheapest possible bench: one machine, no hardware, reproducible in a test script.

```
ip netns add low
ip netns add high
ip link add veth-low type veth peer name veth-high
ip link set veth-low  netns low
ip link set veth-high netns high

ip -n low  addr add 192.168.10.1/24 dev veth-low
ip -n high addr add 192.168.10.2/24 dev veth-high
ip -n low  link set veth-low  up ; ip -n low  link set lo up
ip -n high link set veth-high up ; ip -n high link set lo up

# Static neighbours: no ARP anywhere.
LMAC=$(ip -n low  link show veth-low  | awk '/link\/ether/{print $2}')
HMAC=$(ip -n high link show veth-high | awk '/link\/ether/{print $2}')
ip -n low  neigh replace 192.168.10.2 lladdr "$HMAC" dev veth-low  nud permanent
ip -n high neigh replace 192.168.10.1 lladdr "$LMAC" dev veth-high nud permanent

# Apply the per-side rulesets from above.
ip netns exec low  nft -f /etc/nftables-low.conf
ip netns exec high nft -f /etc/nftables-high.conf

# Run the daemons.
ip netns exec high ./bin/<receiver> ...   # see section 7
ip netns exec low  ./bin/<sender>   ...
```

Namespaces add the interface-scoped rules, which is what you want when testing the firewall policy itself rather than the codec.

4.7 A unidirectional layer-2 tunnel with TAP

Everything so far filters at layer 3 and relies on a ruleset staying in place. There is a better software construction: bridge a TAP interface to each of two *real* Ethernet segments and forward frames between the taps with a program that has no reverse code path. The result is a unidirectional layer-2 tunnel — the same Ethernet semantics as the optical diode of §5, with software standing in for the missing transmitter.

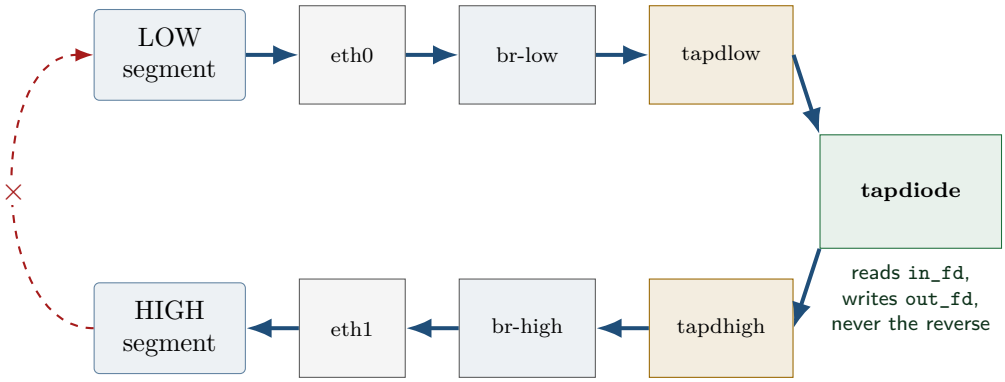

Figure 2: A unidirectional layer-2 tunnel. Each tap is bridged to a real NIC, so two physical Ethernet segments are joined — in one direction.

4.7.1 Why this is a stronger software construction

A firewall diode is a *policy* statement: the path exists and a rule forbids it. This is a *structural* one: the program holds two file descriptors, reads only the first and writes only the second. The reverse direction is not disabled or filtered — it is absent. Nothing to flush, nothing to

misconfigure, and reviewing it means reading eighty lines rather than auditing a ruleset against a kernel's conntrack behaviour.

It is still software, and still not a security boundary

The binary can be replaced, `ptrace`d, or simply bypassed by adding both NICs to one bridge — all of which are one command for root on the forwarding host. It belongs in the same tier as §4: excellent for a bench, for CI, and for teaching the layer-2 semantics of a diode. When the guarantee has to survive a compromise of the host, only optics do that.

4.7.2 The forwarder

The forwarder is about a hundred lines. It attaches to two existing TAP devices, then copies frames from the first to the second for as long as it runs. Table 2 covers the choices in it that are not obvious from the source.

```c
/* tapdiode -- a layer-2 one-way bridge between two TAP interfaces.
 *
 * Frames read from the LOW tap are written to the HIGH tap.  The reverse
 * direction is not disabled, commented out, or filtered: it is absent.
 * out_fd is never read from and never appears in a pollfd, so there is no
 * code path along which a frame could travel HIGH -> LOW.
 *
 * Both taps must already exist and be bridged to different Ethernet
 * segments -- see the bridge setup below.
 */
#define _GNU_SOURCE
#include <errno.h>
#include <fcntl.h>
#include <poll.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/ioctl.h>

#include <net/if.h>          /* IFNAMSIZ, struct ifreq -- NOT linux/if.h */
#include <linux/if_tun.h>

#define FRAME_MAX 65536      /* jumbo frame + VLAN tags, with room to spare */

static volatile sig_atomic_t stop_requested;

static void on_signal(int sig) { (void) sig; stop_requested = 1; }

/* Attach to the existing TAP device 'name'.  Returns a descriptor, or -1. */
static int tap_attach(const char *name)
{
    struct ifreq ifr;
    size_t len = strlen(name);
    int fd;

    /* ifr_name is IFNAMSIZ bytes; validate before copying into it. */
    if (len == 0 || len >= IFNAMSIZ) {
        fprintf(stderr, "tapdiode: bad interface name '%s' (1..%d chars)\n",
                name, IFNAMSIZ - 1);
        return -1;
    }

    fd = open("/dev/net/tun", O_RDWR | O_CLOEXEC);
    if (fd < 0) { perror("tapdiode: open /dev/net/tun"); return -1; }

    memset(&ifr, 0, sizeof ifr);
    /* IFF_TAP   -- layer 2; the buffer is a whole Ethernet frame.
     * IFF_NO_PI -- no 4-byte struct tun_pi prefix to skip. */
    ifr.ifr_flags = IFF_TAP | IFF_NO_PI;
    memcpy(ifr.ifr_name, name, len + 1);

```

```c
    if (ioctl(fd, TUNSETIFF, &ifr) < 0) {
        fprintf(stderr, "tapdiode: TUNSETIFF %s: %s\n", name, strerror(errno));
        close(fd);
        return -1;
    }
    return fd;
}

int main(int argc, char **argv)
{
    static unsigned char frame[FRAME_MAX];
    unsigned long long forwarded = 0, bytes = 0, lost = 0;
    struct sigaction sa;
    struct pollfd pfd;
    int in_fd, out_fd;

    if (argc != 3) {
        fprintf(stderr, "usage: %s <low-tap> <high-tap>\n"
                        "  Forwards Ethernet frames low -> high only.\n", argv[0]);
        return 2;
    }

    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_signal;
    if (sigaction(SIGINT, &sa, NULL) < 0 || sigaction(SIGTERM, &sa, NULL) < 0) {
        perror("tapdiode: sigaction");
        return 1;
    }

    if ((in_fd = tap_attach(argv[1])) < 0) return 1;
    if ((out_fd = tap_attach(argv[2])) < 0) { close(in_fd); return 1; }

    fprintf(stderr, "tapdiode: %s -> %s, one way (no reverse code path)\n",
            argv[1], argv[2]);

    /* The input descriptor is the only thing ever polled, and only for
     * readability.  out_fd is write-only by construction. */
    pfd.fd = in_fd;
    pfd.events = POLLIN;

    while (!stop_requested) {
        ssize_t n, w;

        if (poll(&pfd, 1, -1) < 0) {
            if (errno == EINTR) continue;
            perror("tapdiode: poll");
            break;
        }

        n = read(in_fd, frame, sizeof frame);
        if (n < 0) {
            if (errno == EINTR || errno == EAGAIN) continue;
            perror("tapdiode: read");
            break;
        }
        if (n == 0) continue;

        w = write(out_fd, frame, (size_t) n);
        if (w < 0) {
            if (errno == EINTR) continue;
            /* A full queue downstream is loss, which is the honest failure
             * mode of a diode.  Count it; never block, never signal back. */
            lost++;
            continue;
        }
        if (w != n) { lost++; continue; }   /* short write: frame malformed */
        forwarded++;
        bytes += (unsigned long long) n;
    }

    fprintf(stderr, "tapdiode: forwarded %llu frames (%llu bytes), lost %llu\n",
            forwarded, bytes, lost);
    close(in_fd);
```

```
    close(out_fd);
    return 0;
}
```

Listing 12: `tapdiode.c` — a one-way layer-2 forwarder. Build with `cc -O2 -Wall -Wextra -pedantic -std=c11 -o tapdiode tapdiode.c`

Choice	Reason
Only `in_fd` is polled	`out_fd` never appears in a `pollfd` and `read()` is never called on it, so one-wayness is a property of the control flow — visible to anyone reading the loop, and not dependent on configuration.
`poll()` rather than `select()`	No `FD_SETSIZE` ceiling, no `fd_set` to rebuild each iteration, and the intent — one descriptor, readability only — is explicit in the call.
`IFF_NO_PI`	Suppresses the 4-byte `struct tun_pi` prefix, so the buffer holds exactly the Ethernet frame and nothing has to be skipped on either side.
Interface names as arguments	Bridge membership and addressing can be scripted deterministically instead of depending on which `tapN` the kernel happens to assign.
Name length checked against `IFNAMSIZ`	`ifr_name` is a fixed 16-byte field; the check keeps the copy into it bounded.
`<net/if.h>`, not `<linux/if.h>`	The glibc and kernel headers define overlapping structures; mixing them fails to compile on a current toolchain.
`EINTR` retried; every syscall checked	A signal must not be mistaken for a closed device, and a failed `read()` must never reach `write()` as a length.
Write failure counted, never retried	A full queue downstream is loss, and loss is the honest failure mode of a diode. Blocking or signalling upstream would both be worse.
Counters, not a per-frame `printf`	Per-frame output would dominate the runtime; totals are reported once, on exit.
`O_CLOEXEC`, signal handling, 64 KiB buffer	Descriptors do not leak across an `exec`; `SIGINT`/`SIGTERM` produce a clean summary; jumbo frames and VLAN tags fit with room to spare.

Table 2: Design notes for Listing 12.

4.7.3 Bridging it to real interfaces

```
# One tap per side.  Persistent, so they can be bridged before tapdiode runs.
ip tuntap add dev tapdlow  mode tap
ip tuntap add dev tapdhigh mode tap

# One bridge per side.  STP off: BPDUs are bidirectional signalling and have
# no business on a diode.
ip link add br-low  type bridge stp_state 0
ip link add br-high type bridge stp_state 0

ip link set eth0     master br-low     # NIC facing the SOURCE segment
ip link set tapdlow  master br-low
ip link set eth1     master br-high    # NIC facing the DESTINATION segment
ip link set tapdhigh master br-high

for i in eth0 eth1 tapdlow tapdhigh br-low br-high; do ip link set "$i" up; done

# The forwarding host is a pure layer-2 device: give it no address on either
# side, and no IPv6, so it is not itself reachable from either segment.
for i in br-low br-high eth0 eth1 tapdlow tapdhigh; do
    sysctl -qw "net.ipv6.conf.$i.disable_ipv6=1"
done

./tapdiode tapdlow tapdhigh
```

Listing 13: On the forwarding host

The two segments then carry the addressing, and because this is a layer-2 tunnel the hosts on either side are configured exactly as for the optical diode: same subnet, static neighbour entries, no ARP (§4.2).

Two bridges, never one

The whole construction rests on `br-low` and `br-high` being separate bridges. Adding `eth0` and `eth1` to the *same* bridge produces an ordinary bidirectional bridge and silently destroys the property — and it is a single command. If you script this, assert the membership afterwards: `bridge link show` must never list `eth0` and `eth1` under one master.

Frames from HIGH do enter tapdhigh, and stop there

`br-high` forwards DESTINATION traffic toward `tapdhigh`, so those frames land in that tap's transmit queue — where nothing ever reads them, because `tapdiode` never calls `read()` on `out_fd`. The queue fills and the kernel discards them. This costs a little memory and no security: a tap's transmit and receive queues are independent, so an unread queue cannot exert backpressure on the writes going the other way. If the waste offends, drop the traffic at the bridge with an nftables `bridge`-family rule on egress toward `tapdhigh`.

4.7.4 Verifying it

Layer 2 needs a layer-2 test; `ping` proves nothing about frames the IP stack never generates. Inject a tagged frame with a distinctive EtherType on one segment and sniff for it on the other:

```
# On the DESTINATION segment: capture (EtherType 0x88b5 is reserved for local use)
tcpdump -ni eth0 -e 'ether proto 0x88b5'

# On the SOURCE segment: inject one frame
python3 - <<'PY'
import socket
s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW); s.bind(('eth0', 0))
s.send(b'\xff'*6 + b'\x02\x00\x00\x00\x00\x01' + b'\x88\xb5' + b'DIODETEST')
PY
```

The frame must appear on the DESTINATION segment. Then reverse the two commands: injecting on DESTINATION must produce *nothing* on SOURCE, while a capture on `br-high` confirms the frame really was injected — so that a silent SOURCE means one-wayness and not a broken injector.

In practice

Sniff with `socket.htons(3)` if you write the capture side in Python. The `AF_PACKET` protocol argument is in network byte order and Python does not convert it, so a literal `3` binds to EtherType `0x0300` and sees nothing — a mistake that looks exactly like a failed diode.

5 Hardware diodes

5.1 What a media converter is, and which ones work

A media converter bridges copper Ethernet and optical fiber. Internally it is two transceivers back to back: copper in → fiber out, fiber in → copper out. The optical side is what interests us, and only one kind of optical side can be used.

Optical interface	Usable?	Why
Dual-fiber, discrete TX and RX ports (two SC or two LC ferrules)	**Yes**	TX and RX are physically separate. You can connect one and not the other — which is the entire mechanism of an optical diode.
Single-fiber bidirectional (BiDi / WDM, one strand, e.g. 1310 nm out and 1550 nm back)	**No**	Both directions share one connector and one strand, separated only by wavelength inside the module. There is no TX-only path to expose.

Table 3: Only two-strand converters with separate TX and RX ports can build a diode. Single-port BiDi models cannot, at any price.

The obstacle is that a converter with nothing on its RX port sees no carrier, declares the fiber link down, and stops forwarding. The paper records this directly: converters with a disconnected RX port "will automatically turn off unless electronic hacks are in place" [1]. So a diode is not built by omitting a cable. It is built by feeding the transmitter light that carries no information from the far side. There are exactly two ways to do that, and they are the next two subsections.

5.2 Topology A — two converters and a fiber splitter

Split the transmitter's own outgoing light and feed half of it back into the transmitter's own receiver. The transmitter sees carrier — its own — and holds the link up. The other half of the light is the data path.

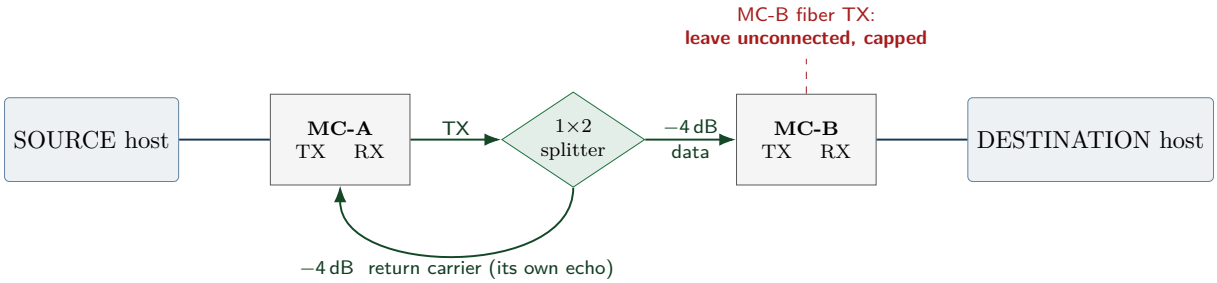

Figure 3: Two-converter diode. The splitter is passive and reciprocal, so the unused legs matter as much as the used ones.

A splitter is reciprocal

An optical splitter is a passive, bidirectional device. Light entering any leg emerges from the others. In Figure 3 the only reason no information flows backwards is that **MC-B's transmitter is not connected to anything**. Cap that port. If someone later patches MC-B's TX into a spare splitter leg — or into the return fiber — DESTINATION data reaches MC-A's receiver and the diode is gone, with no visible change to the front panel LEDs.

The transmitter hears its own echo

MC-A receives its own transmission, so MC-A's copper port re-emits every frame the SOURCE host sent. This is harmless to the diode property — nothing from DESTINATION is involved — but it has two practical consequences:

- **Never put a switch between the source host and MC-A.** The echo becomes a forwarding loop and then a broadcast storm. Connect the host's NIC directly.
- Disable ARP and IPv6 on the diode interface (§4.2) so the host does not react to its own reflected broadcasts.

Topology B does not have this behaviour, which is a quiet point in its favour.

5.3 Sourcing the splitter, and the power budget

The part itself is unremarkable: an ordinary passive 1×2 50:50 splitter. It works with all three converters below, including the 850 nm multimode MC200CM.[1] Topology A is not limited by physics on any of them.

What differs between wavelengths is **availability and price**, and the gap is large. Splitters at 1310 nm are a fiber-to-the-home product — single-mode, specified across 1260 nm–1650 nm, manufactured by the million, stocked everywhere and cheap. Multimode splitters at 850 nm are a specialty item: the 850 nm splitter used for the gigabit build here had to be made to special order and took months to arrive. The component works perfectly well once you have it; the difficulty is entirely in getting one.

What this means for a build

Choose Topology A freely at 1310 nm (MC100CM or MC210CS) — the splitter is a catalogue part you can have tomorrow. At 850 nm (MC200CM) Topology A works just as well, but budget for a special order and a long lead time. If you need the diode sooner than the splitter can arrive, Topology B (§5.4) needs no splitter at all and uses a third converter you can buy off the shelf. Match the splitter's fiber type to the link: single-mode splitter on single-mode fiber, multimode on multimode.

5.3.1 Power budget and attenuation

A 50:50 split costs $10 \log_{10} 2 = 3.01$ dB per leg by construction, plus a little excess loss and connector loss — call it 4 dB in round numbers. Every class below has room for that, which is why the splitter build works across the range. The figures are worth having anyway, because they tell you when to *reduce* the light rather than worry about losing it:

Model	Class	λ	TX min	TX max	RX sens.	RX max	Budget
MC100CM	100BASE-FX	1310 nm	-20 dBm	-14 dBm	-29 dBm	-14 dBm	≈9 dB
MC200CM	1000BASE-SX	850 nm	-9.5 dBm	0 dBm	-17 dBm	0 dBm	≈7.5 dB
MC210CS	1000BASE-LX	1310 nm	-11 dBm	-3 dBm	-19 dBm	-3 dBm	≈8 dB

Table 4: Figures for the *optical class*, not for these units: **TP-Link publishes no optical power data for the MC100CM, MC200CM or MC210CS** — their datasheets give only wavelength, fiber type and distance. These are the published class specifications [5]; a particular module often does better. "Budget" is worst-case launch power minus worst-case receiver sensitivity. All three have room for a ≈4 dB splitter.

Note the pattern in the two maximum columns: in every class the maximum launch power and the maximum receive power are the same number (-14 dBm, 0 dBm, -3 dBm). That is deliberate, and it is what the attenuation advice below turns on.

[1] The *Sensors* paper states that this approach "did not work" for the MC200CM, and reports the three-converter arrangement being used instead ([1], §2.2). That wording describes the parts available at the time rather than a limitation of the method: no 850 nm multimode splitter could be obtained within the timeframe of that work. The one eventually used had to be made to special order and took months to arrive, and with it in hand the two-converter topology works on the MC200CM exactly as it does at 1310 nm.

When to attenuate

A receiver has a maximum input power as well as a sensitivity floor, and exceeding it saturates the front end and raises the bit error rate — which reaches the application as CRC failures and dropped frames.

Within one optical class this is designed out: maximum launch power equals maximum receive power in every class in the table above (-14 dBm for 100BASE-FX, 0 dBm for 1000BASE-SX, -3 dBm for 1000BASE-LX), so a compliant transmitter cannot overload a compliant receiver of the same class even over a metre of fiber. Three matched converters on a bench need no attenuation.

The case that does need it is the paper's: a *long-range* converter or SFP — one built to drive tens of kilometres, and launching accordingly — used across the short fiber a diode actually needs [1]. Fixed attenuators (3 dB, 5 dB, 10 dB) are inexpensive; size them from the launch power and the receiver's maximum in your datasheets rather than by trial. They are also how you trim one leg of a splitter without touching the other.

5.4 Topology B — three converters in a triangle

No splitter to source, and no echo. A third converter acts purely as a carrier source. Its copper port is left unplugged, so it emits an idle optical carrier and can never carry data.

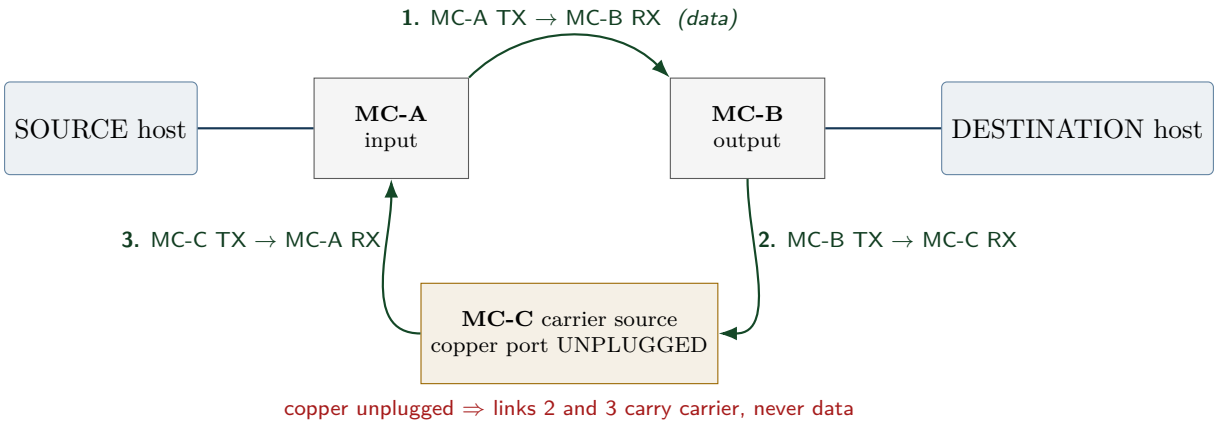

Figure 4: Three-converter diode. The security argument rests on one unplugged copper port.

Why it is safe. Trace a hypothetical leak. DESTINATION data enters MC-B's copper port, leaves MC-B's fiber TX on link 2, and arrives at MC-C's fiber RX. MC-C converts it to copper — and its copper port goes nowhere, so the frames are discarded. MC-C's fiber TX is driven by MC-C's *copper receiver*, which has no link and no traffic, so link 3 carries idle carrier only. The chain is broken at MC-C's unplugged copper port, and nowhere else.

Why link 2 exists at all. MC-C needs carrier on its own RX before it will bring its optical side up and start transmitting. Link 2 supplies it. The paper states the same requirement: the connection from the output converter to the signal emulator "is necessary in order to keep the signal emulator running" [1].

The single point of failure is a copper port

Everything rests on MC-C's copper port being unplugged. Cap it, label it *"MUST REMAIN DISCONNECTED — DATA DIODE"*, and put it first on the audit checklist in §6. A well-meaning administrator patching that port into the SOURCE switch converts the diode

into an ordinary bidirectional link, and no LED changes colour to say so.

If MC-C will not light up

Some converters implement link-fault pass-through and go dark on fiber when their copper link is down. If MC-C behaves that way, give it a copper link that leads nowhere: an isolated switch port on a dead VLAN with no uplink, or a loopback plug. Never a port on the SOURCE network — that would rebuild the return path you just broke.

5.5 Choosing between them

Topology B is the default. It costs one more converter and one more power Both work at every wavelength here. Topology A is the tidier build — one box fewer, one power supply fewer, and a passive component in place of an active one. Topology B trades that for parts you can buy off the shelf anywhere, and it avoids the transmitter echo. At 1310 nm the choice is genuinely free; at 850 nm it comes down to whether you can wait for the splitter.

	A: 2 converters + splitter	**B: 3 converters**
Parts	2 converters, 1 splitter, 3 patch cables	3 converters, 3 patch cables
Power supplies	2	3
Optical margin	Reduced by ≈4 dB; within budget on all three models	Full budget on every link
Procurement	Trivial at 1310 nm; the 850 nm splitter is a special order with a long lead time	Everything is catalogue stock at any wavelength
Transmitter echo	Yes — host hears itself; no switch on the SOURCE leg	No
Failure points	2 active + 1 passive	3 active
Critical rule	MC-B's TX must stay unconnected	MC-C's copper must stay unplugged
Use when	You have the splitter, and want fewer boxes	You want to build today, at any wavelength

5.6 Power, environment and maintenance

Drawn from the paper's §4.3, which is the most practically useful part of it [1]:

- **Power.** These converters run on 9 V DC via a centre-positive barrel connector; the MC200CM draws about 600 mA. The supplies shipped with them are unremarkable. A single stabilised laboratory supply — the paper uses an XP Power ECE60US09-S, 9 V at 6.67 A — will run six diodes at once and is far better regulated, though it costs an order of magnitude more than a converter.
- **UPS.** A diode is on the critical path for log delivery and draws very little. Put it on a UPS.
- **Cooling.** Passive. Six diodes fit in a 2U case with no fans.
- **Attenuators** for short runs with long-range optics (§5.3).
- **MTBF** is comparable to a network card, but a three-converter diode has three of them plus their supplies. Keep a spare set.
- **Latency** is negligible — nanoseconds. It never appears in end-to-end measurements.
- **Managed converters** offer SNMP, fault alerts and test modes, and are a genuine security risk on a diode: a management plane is a second network path into the boundary device. The paper deliberately uses unmanaged units. If you use managed ones, be certain their

management interfaces are on an isolated VLAN, and understand that you have added software to a control whose entire value was that it had none.

5.7 Scaling

Diodes are cheap, so scale out rather than up. Run several in parallel between the same pair of networks, using separate NICs or IP aliases on both sides, and spread messages across them round-robin — the paper's Figure 10(a) arrangement. The receiver binds the same port on every address, so the links aggregate transparently. This is also the answer when the paper's repeat factor would otherwise need to exceed about 50: add a second diode rather than more redundancy on one.

5.8 Embedded scale: a three-wire SPI diode

Media converters are the right answer between two networks. Between two microcontrollers — a sensor gateway that must report into a protected monitoring domain, a safety PLC that must emit telemetry but never accept a command — they are absurd: three converters, three power supplies and fiber patch cords to join two boards that sit ten centimetres apart.

At that scale the diode is three wires. SPI already keeps its two data directions on *physically separate conductors*, so removing one of them removes a direction. This is the same argument as the optical diode, at chip scale and for a few cents.

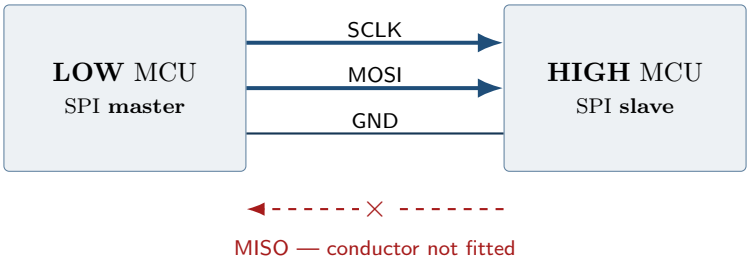

Figure 5: A three-wire SPI diode. Every conductor is driven by the SOURCE side and read by the DESTINATION side; the return data line is not fitted.

5.8.1 The sender must be the master

This is the design decision that matters, and it is not the obvious one.

An SPI master drives SCLK, CS and MOSI, and reads MISO. A slave drives MISO and reads the rest. For information to flow SOURCE → DESTINATION with *no* conductor driven from DESTINATION, the SOURCE side must be the master: it drives clock and data, and the DESTINATION side only ever listens.

The tempting alternative — make the DESTINATION side the master so it "pulls" data — quietly reintroduces a channel. A master drives the clock, so SCLK would run DESTINATION → SOURCE, and the DESTINATION side could then modulate the timing or gating of clock edges into a signal the SOURCE side can observe. It is a low-rate channel, but it is a real one, and it exists by construction rather than by bug.

"3-wire SPI mode" in a datasheet usually means the opposite

Many MCU and sensor datasheets use "3-wire SPI" for *half-duplex* SPI, in which MOSI and MISO are collapsed onto one **bidirectional** data pin (often `SDIO` or `SDA`). That is a shared, driven-from-both-ends conductor — precisely what a diode must not have. What is wanted here is ordinary full-duplex SPI with the MISO wire physically absent. Read the pin direction, not the marketing name.

Signal	Fitted?	Direction and role
SCLK	yes	SOURCE → DESTINATION. Clock; the sender sets the pace.
MOSI	yes	SOURCE → DESTINATION. The data.
GND	yes	Common return. See the isolation note below.
CS	optional	SOURCE → DESTINATION. A fourth conductor that gives hardware frame boundaries; strongly recommended.
MISO	**no**	Would be DESTINATION → SOURCE. Omit the conductor entirely — do not merely leave it unconfigured.

Table 5: Three conductors, or four with CS. The diode property is the row that is not there.

Never add a ready line

Without MISO there is no flow control: the master clocks bits out whether or not the slave is keeping up. The standard embedded fix is a GPIO from slave to master meaning "ready" or "busy" — and that is a wire from DESTINATION to SOURCE. Adding it destroys the diode as thoroughly as fitting MISO would, and it is the single most likely mistake in this design because every SPI tutorial recommends it. Handle overrun the way the rest of this handbook handles loss: clock slowly enough that the receiver always keeps up, use DMA with double buffering, and send redundancy in advance.

5.8.2 Isolation

A shared ground is still a conductor between the two domains. Where that matters — different power domains, an untrusted SOURCE board, or simply a wish to make the property auditable at a glance — put a digital isolator or an optocoupler on each of the three signals. This is worth doing for its own sake:

- An optocoupler is an LED illuminating a phototransistor. It is *physically* incapable of conducting backwards, so each isolated line enforces its own direction in hardware — the optical diode argument, in a DIP-4 package.
- Galvanic isolation removes the shared ground return, so the two domains no longer share a reference at all.
- Check the part's data rate. Ordinary optocouplers manage a few hundred kbit/s; use a proper digital isolator for anything faster, and confirm it is a single-channel unidirectional device rather than a bidirectional or mixed-direction part.

5.8.3 Framing, and what to run over it

With CS fitted, the slave gets frame boundaries in hardware and resynchronizes for free. Without it the receiver must find its own boundaries in the bit stream, so frame in band: a sync word, a length, the payload, and a check value. That is the same bounded-cursor discipline as everywhere else in this system — a length is validated against what remains before it is ever used as an index, and a length that lies ends the frame instead of becoming a pointer.

Everything the rest of this handbook says about a feedback-free link applies unchanged: no acknowledgement, so no retransmission; redundancy must be sent ahead of time; and authentication is what distinguishes a corrupted frame from a forged one.

Reusing the proven core on an MCU

The SPARK units in `src/core` are fixed-capacity and allocation-free by construction, so they do not depend on an operating system and can be cross-compiled for a bare metal target. `od_sender` and `od_receiver` themselves cannot: they are Linux daemons built on UDP

sockets, and the sockets would have to be replaced by SPI transfers.
The capacities are the obstacle worth planning for. As analyzed, the instance assumes `Secure.Max_Plain` = 65536, `Syslog.Max_Record` = 65535 and a relay holding sixteen in-flight messages — megabytes of static buffers, which no small microcontroller has. Reducing them is a supported change, but it changes what was proved: the capacities are part of the analyzed instance, so a shrunken build must be re-proved before any of the assurance claims in §2 carry over to it.

5.8.4 Commissioning an SPI diode

1. **Count the wires.** Three, or four with CS. This is the whole audit, and it is why the design is attractive — correctness is visible without instruments.
2. **Continuity.** With both boards powered down, confirm an open circuit between the DESTINATION MCU's MISO pin and every conductor in the cable. Repeat after any rework.
3. **Drive test.** Configure the DESTINATION MCU to drive its MISO pin hard, high and low, in a loop. The SOURCE MCU must see nothing on any pin. This catches a MISO track left on a shared PCB, which a wire count will not.
4. **Isolator orientation.** If optocouplers are fitted, confirm each one's input side faces SOURCE. A part installed backwards will usually appear to work — in the wrong direction.
5. **Overrun.** Run the sender at full rate for an extended period and confirm the receiver's dropped-frame counter behaves as loss, not as corruption reaching the application.

6 Commissioning and verification

A diode you have not tested is a diode you have assumed. The tests below are ordered so that each one fails cheaply before the next.

6.1 Physical audit

1. Trace every fiber by hand against the wiring table for your topology.
2. Topology B: confirm MC-C's copper port is unplugged and capped. Topology A: confirm MC-B's fiber TX is unconnected and capped.
3. Confirm the SOURCE host connects *directly* to MC-A, with no switch (mandatory for Topology A because of the echo).
4. Photograph the finished wiring. Attach it to the commissioning record; it is what a future auditor compares against.

6.2 Carrier independence

Unplug MC-B's copper cable from the DESTINATION host. MC-A must keep its fiber link LED. If MC-A drops, its carrier is arriving from the far end and the link is bidirectional.

Repeat in the other direction: power MC-C down (Topology B). MC-A should lose its fiber link, confirming that MC-C — and not MC-B — is its carrier source.

6.3 Active reverse probing

Capture everything on the SOURCE host for the duration:

```
# SOURCE host -- runs for the whole test, must stay silent
tcpdump -ni enp1s0 -e -vv -w /tmp/diode-commissioning.pcap
```

From the DESTINATION host, attempt every reasonable back channel:

```
ping   -c 5            192.168.10.1        # ICMP echo
arping -c 5 -I enp1s0  192.168.10.1        # link layer
nc -u -w2              192.168.10.1 9999   # UDP
nc    -w5              192.168.10.1 22     # TCP -- must time out
hping3 -S -c 5 -p 22   192.168.10.1        # raw SYN
hping3 -1 -c 5         192.168.10.1        # raw ICMP
```

Then verify the capture contains nothing whose source is the DESTINATION host:

```
tcpdump -nr /tmp/diode-commissioning.pcap "src host 192.168.10.2"
# must print no packets
```

6.4 Forward path and end-to-end

```
# DESTINATION host
tcpdump -ni enp1s0 udp port 6514
# SOURCE host
echo hello | nc -u -w1 192.168.10.2 6514

# then, with the daemons running:
# SOURCE host
logger -n 127.0.0.1 -P 1514 -p local0.crit "diode commissioning $(date -Is)"
# DESTINATION host
journalctl -f | grep "diode commissioning"
```

Finally, pull MC-A's TX fiber: the DESTINATION side must stop receiving immediately, and the SOURCE side must be entirely unaffected — it has no way to notice.

6.5 Recurring audit

There is no remote diagnostic and no alarm. Failure and tampering are both silent, so both need a calendar entry.

Interval	Check
Continuous	DESTINATION-side monitoring alerts if the log stream stops. This is the only automatic failure indication you get — configure it.
Monthly	Physical inspection: all LEDs as expected, MC-C's copper port still unplugged, seals intact, photo still matches.
Quarterly	Repeat the active reverse probe of §6.3 in full.
On change	Any rewiring, converter replacement, or firmware change re-runs the entire commissioning sequence from §6.1.

7 Running opc-diode across the link

The relay is two programs. Start the receiver first, so nothing is lost.

```
./bin/od_receiver <diode_port> <out_ip> <out_port> [--key HEX64]

# e.g. recover on 9702, re-emit NetworkMessages to subscribers on 127.0.0.1:9703
./bin/od_receiver 9702 127.0.0.1 9703 --key <64 hex>
```

Listing 14: DESTINATION side — recover and re-emit to subscribers

```
./bin/od_sender <in_port> <diode_ip> <diode_port> [--parity M] [--interleave D]
                [--pace-us N] [--key HEX64]
```

```
# e.g. take NetworkMessages on 9701, add 3 parity fragments, blast to the diode
./bin/od_sender 9701 <receiver_ip> 9702 --parity 3 --pace-us 200 --key <64 hex>
```

Listing 15: SOURCE side — take the publisher's output, protect, transmit

The publisher points its PubSub output at the sender's input port; subscribers listen on the receiver's output port. One-way throughout.

7.1 The options that matter

`-parity M` — how much loss you survive.
M parity fragments are added to the K data fragments of a message, and any K of the $K + M$ rebuild it. Raise it first when messages go missing.

`-interleave D` — what *shape* of loss you survive.
Spreads the fragments of D messages in time (packet 1 of each, then packet 2 of each), so a burst becomes a recoverable one-per-message trickle rather than the loss of one whole message. Costs latency, not bandwidth.

`-pace-us N` — how hard you drive the link.
Microseconds between packets. A diode has no flow control, so pacing is what stops the sender overrunning the receiver.

`-key HEX64` — authentication.
ChaCha20-Poly1305, on *both* ends. The sender seals each NetworkMessage before fragmenting; the receiver verifies the tag and drops anything that fails. Without it, payloads cross in the clear. The key is pre-shared out of band — see the note in §2 on why no key agreement is possible here.

opc-diode also offers a DPDK kernel-bypass transport (`-with-dpdk` and `-eal`); see the project README, which is the authority on the CLI. The assurance argument, including what is proved and what is trusted, is in `docs/ASSURANCE.md`.

8 Troubleshooting

Symptom	Likely cause	Check
No fiber link LED on MC-A	No carrier on its RX	Topology B: is MC-C powered and lit? Is link 3 seated? Topology A: is the return leg of the splitter connected?
MC-C completely dark (Topology B)	Link-fault pass-through, copper down	Give MC-C a dead-VLAN copper link or loopback plug (§5.4).
Link comes up, then flaps	Contamination, a bad splice, or a marginal connector	Clean and reseat the ferrules first — this is the usual cause by a wide margin. Only suspect overload if you are running long-range optics over a short fiber (§5.3).
DESTINATION receives nothing, LEDs all good	ARP or firewall	`ip neigh show` must report PERMANENT. Check `rp_filter` if the two sides are on different subnets. `tcpdump` on the DESTINATION NIC.
DESTINATION sees datagrams but produces no output	Tag verification failing	Keys differ between the two sides, or one side has `-key` and the other does not.

Symptom	Likely cause	Check
High loss under load	Receiver overrun	Raise `-parity`, add `-pace-us`, raise the socket receive buffer, or add a second diode in parallel.
Loss in bursts, fine otherwise	No interleaving	Raise `-interleave`; this is the case it exists for.
Whole groups of lines vanish together	Batch coupling	A failed block loses all N records. Lower `-batch` or raise `-parity`.
Broadcast storm on the SOURCE segment	Topology A echo through a switch	Connect the SOURCE host directly to MC-A (§5.2).
SOURCE host logs traffic from itself	Topology A echo, expected	Harmless. Confirm the source is the SOURCE host, not DESTINATION — if it is DESTINATION, stop and re-run §6.

References

[1] A.-A. Anton, P. Csereoka, E. A. Capotă and R.-D. Cioargă, "Enhancing Syslog Message Security and Reliability over Unidirectional Fiber Optics", *Sensors*, vol. 24, no. 20, 6537, 2024. `https://doi.org/10.3390/s24206537`

[2] D. E. Bell and L. J. LaPadula, "Secure Computer System: Unified Exposition and Multics Interpretation", MITRE Technical Report ESD-TR-75-306, 1976.

[3] R. Gerhards, "The Syslog Protocol", RFC 5424, IETF, 2009.

[4] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols", RFC 8439, IETF, 2018.

[5] IEEE Std 802.3, *IEEE Standard for Ethernet*: clause 26 and the FDDI PMD it adopts (100BASE-FX), and clause 38 (1000BASE-SX, 1000BASE-LX). Source of the per-class optical figures in §5.3. Those figures are class specifications: the vendor datasheets for the MC100CM, MC200CM and MC210CS give wavelength, fiber type and distance only, and publish no optical power data at all.

[6] M. Guri, "ETHERLED: Sending Covert Morse Signals from Air-Gapped Devices via Network Card (NIC) LEDs", *IEEE International Conference on Cyber Security and Resilience (CSR)*, 2022, pp. 163–170.

[7] M. Guri, "LANTENNA: Exfiltrating Data from Air-Gapped Networks via Ethernet Cables Emission", *IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*, Madrid, 2021, pp. 745–754.

[8] M. Guri, B. Zadov, D. Bykhovsky and Y. Elovici, "PowerHammer: Exfiltrating Data From Air-Gapped Computers Through Power Lines", *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 1879–1890, 2020.

[9] M. Guri, A. Kachlon, O. Hasson, G. Kedma, Y. Mirsky and Y. Elovici, "GSMem: Data Exfiltration from Air-Gapped Computers over GSM Frequencies", *24th USENIX Security Symposium*, Washington DC, 2015, pp. 849–864.

[10] R. Chapman, *SPARKNaCl*: a verified Ada/SPARK implementation of TweetNaCl. `https://github.com/rod-chapman/SPARKNaCl`

[11] opc-diode project, *Trusted-boundary assurance argument*, `docs/ASSURANCE.md`.